

Configuración de Servicios: Servidor de Archivos Seguro (SFTP)

Parte 1: Fundamentos y Arquitectura

1. FTP (File Transfer Protocol)

Generalmente cuando se estudia administración de sistemas, se suele enseñar el despliegue de servidores FTP utilizando daemons como vsftpd o proftpd. Pero, en la actualidad, desplegar un FTP para transferencias autenticadas es considerado una vulnerabilidad enorme por dos razones de arquitectura:

A. La Transmisión en Texto Plano

FTP fue diseñado en 1971, una época donde la seguridad no era una prioridad. Todo el tráfico, desde el usuario y la contraseña (comandos USER y PASS) hasta el contenido de los archivos físicos, viaja sin ningún tipo de cifrado. Cualquier atacante posicionado en la misma red local (LAN) puede utilizar un sniffer (como Wireshark) y capturar las credenciales de los administradores sin ningún tipo de inconvenientes..

B. El problema del Ruteo (Modo Activo vs. Pasivo)

A diferencia de servicios más actuales (como HTTP) que abren un único túnel bidireccional, FTP requiere negociar dos conexiones TCP separadas:

1. El Canal de Control (Puerto 21): Para enviar comandos (ej. listar directorio, loguearse).
2. El Canal de Datos: Para mandar los archivos físicos.

El problema es cómo se abre ese segundo canal:

~ En Modo Activo: El servidor intenta iniciar una conexión hacia la máquina del cliente. Esto falla el 99% de las veces en la actualidad porque los firewalls y los routers NAT de los clientes bloquean las conexiones entrantes.

~ En Modo Pasivo: El servidor le dice al cliente "Abrí un puerto aleatorio de mi lado (ej. el puerto 45032) y conectate ahí". Esto soluciona el problema del cliente, pero obliga al SysAdmin a dejar miles de puertos dinámicos abiertos en el firewall de la empresa, que sería un problema enorme desde el punto de vista del hardening.

2. La Evolución: SFTP (SSH File Transfer Protocol)

Para solucionar los problemas de ruteo y seguridad, se migró al protocolo SFTP.

Es importante recalcar que SFTP no es FTP. No comparten código, ni comparten arquitectura. SFTP es un subsistema que corre montado íntegramente sobre el túnel criptográfico de Secure Shell (SSH).

Ventajas de SFTP:

1. Cifrado Absoluto: Al correr dentro de SSH, tanto las credenciales como los archivos viajan protegidos por criptografía asimétrica (llaves) y simétrica (AES), volviendo inútil cualquier intento de interceptación en la red.
2. Single-Port (Multiplexación): Elimina el dolor de cabeza del modo Activo/Pasivo. Todo el control y los datos viajan por un único puerto (el TCP 22 de SSH). Al firewall de la empresa solo hay que abrirle un agujero, no miles.
3. No requiere demonios extra: No hay que instalar vsftpd ni gastar RAM en procesos nuevos. El propio motor de OpenSSH (sshd) que ya tenemos asegurado trae el módulo de SFTP incorporado.

3. La Jaula (Chroot Jail)

El único peligro de usar SFTP es que, como usa la infraestructura de SSH, si le damos un usuario y clave a un cliente para que suba sus archivos web, ese cliente técnicamente podría intentar abrir una terminal y empezar a ver en la raíz de nuestro servidor (/etc/, /var/).

Para evitar esto, en la práctica de laboratorio vamos a implementar una política de Chroot Jail (Enjaulamiento) directo en el archivo sshd_config. Básicamente le vamos a decir al Kernel: "Cuando este usuario se conecte, obligalo a usar solo SFTP, no le des acceso a la terminal, y hacele creer que su carpeta personal es la raíz (/) de todo el disco duro".

Parte 2: Práctica de SFTP Enjaulado (Chroot Jail)

En esta práctica, simularemos la creación de un usuario llamado cliente_web, el cual solo debe tener permisos para subir archivos a su propia carpeta, pero sin ningún tipo de acceso administrativo o de Shell al servidor.

En mi caso, re utilizare la VM que use anteriormente para configurar y hacer hardening del servicio SSH (en el repo de github se encuentra como **07_SSH**).

Paso 1: Creación del entorno aislado.

1. Para esto nos logueamos como root en nuestro servidor

```
> ssh luck4@192.168.122.20
Linux UTN-Lucka 6.12.100+de

The programs included with
the exact distribution term
individual files in /usr/sh

Debian GNU/Linux comes with
permitted by applicable law
Last login: Sat Aug  1 10:5
luck4@UTN-Lucka:~$ su -
Password:
root@UTN-Lucka:~#
```

2. Crear usuario sin shell.

Vamos a crear al usuario cliente_web, pero le vamos a asignar el binario /bin/false como su "intérprete de comandos". Esto es para que si intenta entrar por SSH tradicional para tirar comandos, el sistema lo saque instantáneamente.

```
~ useradd -m -s /bin/false cliente_web  
~ passwd cliente_web
```

```
root@UTN-Lucka:~# useradd -m -s /bin/false cliente_web  
root@UTN-Lucka:~# passwd cliente_web  
New password:  
Retype new password:  
passwd: password updated successfully  
root@UTN-Lucka:~#
```

3. Preparar Jaula Chroot:

Lo mas importante de la configuración del subsistema SFTP en OpenSSH es que el directorio raíz de la jaula debe pertenecer a root y nadie más puede tener permisos de escritura. Si esto no se cumple, el servicio se niega a arrancar por seguridad.

Entonces cambiaremos el propietario de la carpeta personal a root.

```
~ chown root:root /home/cliente_web  
~ chmod 755 /home/cliente_web
```

```
root@UTN-Lucka:~# chown root:root /home/cliente_web  
root@UTN-Lucka:~# chmod 755 /home/cliente_web  
root@UTN-Lucka:~#
```

4. Crear directorio para la subida de archivos (Uploads):

Como le quitamos el permiso de escritura sobre su propio /home para que la jaula funcione, tenemos que crearle una carpeta interna donde sí pueda soltar sus archivos.

```
~ mkdir /home/cliente_web/subidas  
~ chown cliente_web:cliente_web /home/cliente_web/subidas
```

```
root@UTN-Lucka:~# mkdir /home/cliente_web/subidas; chown cliente_web:cliente_web /home/cliente_web/subidas
```

Paso 2: configuracion de SSH (sshd_config)

Ahora vamos a editar sshd_config para habilitar el subsistema de transferencia y enjaular a este usuario específico.

1. Abriremos la configuración:

```
~ vim /etc/ssh/sshd_config
```

2. Habilitar el módulo interno de SFTP

Buscaremos la línea que dice Subsystem sftp y la comentaremos #, abajo habilitaremos el módulo interno: Subsystem sftp internal-sftp

```
# override default of no subsystems
#Subsystem      sftp      /usr/lib/openssh/sftp-server
Subsystem       sftp      internal-sftp

# Example of overriding settings on a per-user basis
#Match User anoncvs
#      X11Forwarding no
#      AllowTcpForwarding no
#      PermitTTY no
#      ForceCommand cvs server
-- INSERT --
```

3. Crear la regla de la Jaula (Bloqueo condicional)

Al final de todo el archivo crearemos una regla de coincidencia (match).

Básicamente le diremos al daemon : "Si el usuario que se conecta es cliente_web, forzale estas restricciones".

En la ultima línea pondremos este bloque:

Match User cliente_web

ChrootDirectory /home/cliente_web

ForceCommand internal-sftp

AllowTcpForwarding no

X11Forwarding no

PasswordAuthentication yes

```
ForceCommand cvs server
AllowUsers luck4 cliente_web
Match User cliente_web
    ChrootDirectory /home/cliente_web
    ForceCommand internal-sftp
    AllowTcpForwarding no
    X11Forwarding no
    PasswordAuthentication yes

"/etc/ssh/sshd_config" 133L, 3636B
```

Explicación del bloque:

- ~ Match User: Aplica solo a este usuario.
- ~ ChrootDirectory: Obliga al sistema a cambiar su raíz (/) a esta carpeta en el momento del login.
- ~ ForceCommand: Ignora cualquier comando que intente mandar el cliente y lo obliga a usar el protocolo SFTP de transferencia.
- ~ Forwarding no: Le prohíbe usar el servidor para hacer túneles de red.
- ~ PasswordAuthentication yes: Como es un cliente externo, le permitimos usar su contraseña temporal (ya que el usuario root general sigue protegido por llaves gracias al hardening global).

Para aplicar los cambios reiniciamos el servicio: ~ systemctl restart sshd

Paso 3: Prueba y validación.

Prueba 1: Intento de acceso interactivo (El bloqueo)

Abriremos una terminal desde nuestra maquina fisica y trataremos de usar comandos

~ **ssh cliente_web@tulP**

El resultado esperado es que nos pida la credencial pero cuando tratemos de ingresar el servidor nos tire la conexión, esto es gracias a el /bin/false y el ForceCommand del sshd_config.

```
> ssh cliente_web@192.168.122.20
cliente_web@192.168.122.20's password:
This service allows sftp connections only.
Connection to 192.168.122.20 closed.
```

Prueba 2: Transferencia Segura (El acceso legítimo)

Desde la misma terminal en la máquina física, nos conectamos usando el cliente SFTP en lugar del cliente SSH:

~ sftp cliente_web@192.168.122.20

El resultado esperado es que nos conectemos con éxito y nos aparecerá el prompt en: *sftp>*
Si usamos el comando *pwd* (Print Working Directory), el servidor nos va a mentir y te va a decir que estamos en / (la raíz), probando que la jaula funciona.

```
> sftp cliente_web@192.168.122.20
cliente_web@192.168.122.20's password:
Connected to 192.168.122.20.
sftp> pwd
Remote working directory: /
sftp> 
```

Podemos usar *put* para copiar archivos y probar el servidor

~ cd subidas

~ put /etc/hosts

(Esto copiará tu archivo *hosts* local hacia la carpeta *subidas* del servidor de forma cifrada).

```
sftp> put /etc/ho
host.conf      hostname      hosts         hosts.allow   hosts.deny
sftp> put /etc/hosts
Uploading /etc/hosts to /subidas/hosts
hosts                                                  100% 192    2.0MB/s   00:00
sftp> 
```

Conclusión:

En este laboratorio, demostramos que la administración actual de infraestructura no se trata simplemente de levantar servicios, sino de comprender sus vulnerabilidades de arquitectura.

Justificamos porque FTP está obsoleto, descartando su uso debido a la transmisión de datos en texto plano y a los problemas de enrutamiento que generan sus modos Activo y Pasivo frente a los Firewalls.

En su lugar, usamos la infraestructura cifrada de OpenSSH para desplegar SFTP (Single-Port). Pero hicimos las buenas practicas aplicando el principio de Confianza Cero (Zero Trust) y el Principio de Menor Privilegio: a través de la configuración de un *Chroot Jail*, logamos que un usuario externo pueda transferir archivos de forma totalmente encriptada, pero sin tener ninguna posibilidad de ejecutar comandos en la terminal ni de navegar por el árbol de directorios del sistema operativo.